Build an Authorization Decorator

Check who is calling before a function runs – login and role rules enforced in one reusable place.

Step 1 – require a logged-in user

```
from functools import wraps

class AuthError(PermissionError):
    pass

def login_required(fn):
    @wraps(fn)
    def wrapper(user, *args, **kwargs):
        if not user.get("authenticated"):
            raise AuthError("login required")
        return fn(user, *args, **kwargs)
    return wrapper

@login_required
def view_profile(user):
    return f"profile of {user['name']}"

ann = {"name": "Ann", "authenticated": True}
print(view_profile(ann))
try:
    view_profile({"name": "Eve"})
except AuthError as e:
    print("denied:", e)
```

OUTPUT

```
profile of Ann
denied: login required
```

Step 2 – role-based access (decorator factory)

```
def requires_role(*roles):
    need = " or ".join(roles)
    def decorator(fn):
        @wraps(fn)
        def wrapper(user, *args, **kwargs):
            if user.get("role") not in roles:
                raise AuthError(f"need {need}")
            return fn(user, *args, **kwargs)
        return wrapper
    return decorator

@requires_role("admin")
def delete_user(user, target):
    return f"{target} deleted by {user['name']}"

admin = {"name": "Ann", "role": "admin"}
guest = {"name": "Gus", "role": "guest"}
print(delete_user(admin, "bob"))
try:
    delete_user(guest, "bob")
except AuthError as e:
    print("denied:", e)
```

OUTPUT

```
bob deleted by Ann
denied: need admin
```

Step 3 – stack them

```
@login_required
@requires_role("admin", "editor")
def publish(user, post):
    return f"{post} published"

ed = {"name": "Ed", "role": "editor",
      "authenticated": True}
print(publish(ed, "post-1"))
```

OUTPUT

```
post-1 published
```

Outer decorator runs first: **login** is checked, then the **role**.

Permissions: deny by default

```
PERMS = {"admin": {"read", "write", "delete"},
         "editor": {"read", "write"},
         "viewer": {"read"}}

def requires_perm(perm):
    def decorator(fn):
        @wraps(fn)
        def wrapper(user, *a, **kw):
            role = user.get("role")
            allowed = PERMS.get(role, set())
            # unknown role => deny
            if perm not in allowed:
                raise AuthError(f"missing {perm}")
            return fn(user, *a, **kw)
        return wrapper
    return decorator

@requires_perm("write")
def edit(user, doc):
    return f"{doc} edited"

print(edit({"role": "editor"}, "doc1"))
try:
    edit({"role": "viewer"}, "doc1")
except AuthError as e:
    print("denied:", e)
```

OUTPUT

```
doc1 edited
denied: missing write
```

In a web framework

```
# Flask-style usage (route outermost)
@app.route("/admin")
@login_required
@requires_role("admin")
def admin_page():
    ...
```

Status	Meaning
401 Unauthorized	not logged in / bad credentials
403 Forbidden	logged in, but not allowed

PITFALL A decorator only guards **server-side** code. Hiding a button is not security: always check on the server, deny by default, log denied attempts, and use a vetted auth library for passwords and tokens.